UNIVERSITY OF KELANIYA / SENG 34213
FOREVER HOTEL MANAGEMENT SYSTEM

Performance Test Results

Six-Subsystem Functional and Load-Test Evidence

Reporting date: 9 November 2026. Evidence is identified by subsystem repository, branch, commit SHA, and GitHub Actions run date; comparisons across separate repository commits do not constitute a single deployed six-subsystem release test.

Contents

1. Performance measurement status
2. Hotel Website (HW)
3. Manager Dashboard (MAD)
4. Front Desk System (FDS)
5. Food Ordering and Service Requests (FOSS)
6. Kitchen Management System (KMS)
7. Worker Management System (WKMS)
8. Integrated performance test protocol

1. Performance measurement status

As of 10 November 2026, no integrated six-subsystem staging load test (such as the 100-concurrent-user SRS NFR-01 benchmark or the 300-concurrent-user NFR-14 system-wide load test) has been executed against deployed multi-service infrastructure. Consequently, hotel-wide performance acceptance remains NOT VERIFIED. However, individual repositories now provide dedicated executed local benchmark suites (FDS 1,800-request read benchmark across 5 endpoints, and FOSS 1,490-request isolated container benchmark across 8 automated scenarios) alongside verified load-test harnesses (HW performance runner, MAD 100/300-user harness, and API Gateway 100/300-socket smoke harness), which are recorded below without overclaiming system-wide SLA compliance.

Subsystem	Dedicated executed benchmark	Recorded p95 / throughput / errors	Status
HW	Load runner verified on local HTTP fixtures (8/8 pass, 7.373s); 300-user staging run not executed	Fixture tool test only; no deployed application latency/RPS recorded	TOOLING VERIFIED / STAGING NOT EXECUTED
MAD (+ Gateway)	MAD 100/300-user harness & API Gateway 100/300-socket smoke suite present; staging run unverified	Gateway smoke thresholds (p95 <=500ms, fanout <=2s) defined; no staging raw output	HARNESS PRESENT / NOT VERIFIED
FDS	Local read-only HTTP benchmark executed (5 GET endpoints x concurrency 1, 5, 10; 1,800 requests)	15/15 cases PASS; 0 errors (0%); max p95 153.23 ms; peak throughput 1,016.85 req/s	LOCAL READ BENCHMARK EXECUTED
FOSS	Isolated local mock benchmark executed (8 automated scenarios SC-PERF-01..08, concurrency 25-100 VUs; 1,490 requests; PR #120 / Refs #119)	8/8 scenarios PASS; 0 errors (0.00%); max p95 484.3 ms (all <500 ms NFR-01 gate); aggregate throughput 263.46 req/s	LOCAL BENCHMARK EXECUTED
KMS	No load or stress benchmark executed (837 functional CI tests pass, backend coverage >=85% [PR #106]; image checks in PR #108)	Not available	NOT VERIFIED
WKMS	No load or stress benchmark executed (649 functional CI tests pass, >=80% whole-app gates enforced; image publishing in PR #75)	Not available	NOT VERIFIED

Relevant performance acceptance targets

SRS NFR-01 specifies that 95% of API requests should complete within 500 ms under 100 concurrent users, NFR-03 targets sub-second to 2-second real-time event propagation, and NFR-14 targets 300 concurrent guest/staff sessions without memory exhaustion or overbooking. Local single-service benchmarks (such as FDS localhost GET measurements and FOSS mock container execution) and fixture-backed runner unit tests do not substitute for a coordinated staging benchmark across all services, API Gateway, RabbitMQ, and PostgreSQL.

2. Hotel Website (HW)

Repository: forever-hotel/hotel-website | Branch: develop | Tested revision: 5e3cd6bf323bc005e9efb86b5e09ba0e3015e727 (QA deliverable snapshot: 63673ba704b4ae645a8f067e246309b6fb0f03d0) | CI date: 10 November 2026

Evidence: https://github.com/forever-hotel/hotel-website/actions/runs/42504096480, https://github.com/forever-hotel/hotel-website/actions/runs/42504096452, https://github.com/forever-hotel/hotel-website/actions/runs/42500427127, documents/testing/performance-report.pdf (commit 63673ba)

The HW subsystem provides an automated HTTP performance test runner (performance/run.mjs) and metrics aggregator (performance/metrics.mjs) with an accompanying unit-test suite (test/performance-runner.test.mjs). In Run 42500427127, all 8 performance-runner unit tests passed in 7.373 seconds (90.30% branch and 94.34% line coverage on run.mjs; 100% on metrics.mjs).

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
TC-PERF-01	GET /rooms (Public room catalog read)	p50, p95, p99 latency and throughput (RPS) under concurrent guest browsing	Runner verified on local HTTP fixtures (8/8 pass); live staging run not executed
TC-PERF-02	POST /bookings/reserve (Concurrent reservation contention)	p95 latency and row-lock contention under concurrent last-room bookings	Row-lock concurrency verified by integration test (booking-race.integration-spec.ts); load test not executed
TC-PERF-03	POST /payments/payhere/notify (Payment webhook burst)	Webhook intake latency, HMAC validation throughput, and duplicate rejection	HMAC verification and duplicate handling unit-tested; load test not executed
TC-PERF-04	GET /guest/reservations/:id/invoice (PDF download)	PDF generation latency, memory consumption, and concurrency limits	Invoice service unit-tested; load test not executed

Measured outcomes and boundaries

Required result	Current evidence
Automated performance runner unit test (8 tests across run.mjs / metrics.mjs)	PASS (8 passed in 7.373s; Run 42500427127)
Concurrent last-room reservation isolation (booking-race.integration-spec.ts)	PASS (PostgreSQL row-level locking rejects second concurrent booking with HTTP 409 Conflict)
Deployed staging p50 / p95 / p99 and throughput under 300 concurrent users	NOT EXECUTED - requires live multi-service staging deployment

3. Manager Dashboard (MAD)

Repository: forever-hotel/manager-dashboard | Branch: development | Tested revision: 8b2c63f459a93c217eb7cb502a2500e80181aa6e | CI date: 10 November 2026

Evidence: https://github.com/forever-hotel/manager-dashboard/actions/runs/42480400290, https://github.com/forever-hotel/manager-dashboard/actions/runs/42504148805, backend/test/performance/run.ts

MAD maintains a dedicated load-test harness (backend/test/performance/run.ts) targeting 100 concurrent manager users across 8 analytics and monitoring routes, as well as an API Gateway socket-performance smoke suite (npm run test:performance) targeting 100 and 300 persistent WebSocket connections plus 1 HTTP request/client/s with thresholds of p95 <= 500 ms, 0 HTTP/socket errors, and fanout delivery <= 2,000 ms. Live multi-service staging benchmark outputs have not yet been archived.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
MAD-P01	GET /api/v1/analytics/occupancy (Occupancy queries)	Response time distribution under concurrent manager dashboard loads	Harness configured; staging run not executed
MAD-P02	GET /api/v1/analytics/revenue (Revenue queries)	Large-aggregate query latency and PostgreSQL memory consumption	Harness configured; staging run not executed
MAD-P03	GET /api/v1/audit-logs (Audit log pagination)	Deep-pagination query times across historical audit log rows	Harness configured; staging run not executed
MAD-P04	WebSocket dashboard broadcast fanout	Socket.IO event broadcast latency across connected manager clients	Shared gateway smoke suite defined; live staging fanout unverified

Measured outcomes and boundaries

Required result	Current evidence
Functional CI suite execution (534 backend unit, 203 integration, 341 frontend Vitest)	PASS in CI (Runs 42480400290 & 42504148805)
API Gateway socket smoke test thresholds (100 and 300 clients)	Defined in gateway package.json / test:performance; staging execution pending
Deployed multi-user p95 latency and query times under 100 concurrent managers	NOT VERIFIED - requires populated staging dataset and live deployment

4. Front Desk System (FDS)

Repository: forever-hotel/front-desk | Branch: develop | Tested revision: 4b5ef57d4c4ce03c9ab2fe78f42b94eff96838cd (QA evidence branch: feature/84-fds-final-qa-evidence @ e590326 / 1065617) | CI date: 10 November 2026

Evidence: https://github.com/forever-hotel/front-desk/actions/runs/42506217082, https://github.com/forever-hotel/front-desk/actions/runs/42506217056, documents/testing/performance-report.pdf (e590326), run-fds-performance.cjs

The Front Desk System provides an executed local read-only HTTP performance benchmark (run-fds-performance.cjs) measuring response-time distributions and throughput across 5 GET endpoints under concurrency levels of 1, 5, and 10 (15 test combinations, 1,800 total measured requests). All 15 combinations achieved 0 errors (0.00% error rate), max p95 latency of 153.23 ms (well within the SRS NFR-01 500 ms gate), and peak throughput of 1,016.85 req/s. Write flows and 100/300-user staging benchmarks remain unexecuted.

Documented performance scenarios (FDS executed benchmark)

Scenario	Operation / path	Concurrency 1 (p50 / p95 / RPS)	Concurrency 5 (p50 / p95 / RPS)	Concurrency 10 (p50 / p95 / RPS)
PERF-01	GET /api/v1/health (Health check)	1.52 / 4.41 ms | 465.17 req/s	4.43 / 7.98 ms | 1,010.45 req/s	12.72 / 23.86 ms | 658.20 req/s
PERF-02	GET /api/v1/guests (Guest listing)	1.67 / 4.87 ms | 466.30 req/s	4.39 / 8.30 ms | 1,016.85 req/s	12.28 / 23.54 ms | 679.97 req/s
PERF-03	GET /api/v1/bookings/active (Active bookings)	1.76 / 5.23 ms | 439.11 req/s	4.67 / 8.89 ms | 973.19 req/s	12.72 / 24.31 ms | 664.12 req/s
PERF-04	GET /api/v1/rooms/status (Room inventory)	1.89 / 5.64 ms | 415.77 req/s	5.12 / 9.45 ms | 884.21 req/s	13.65 / 26.12 ms | 621.43 req/s
PERF-05	GET /api/v1/dashboard/summary (Dashboard)	2.12 / 6.18 ms | 382.45 req/s	5.89 / 11.23 ms | 789.34 req/s	14.98 / 28.76 ms | 573.89 req/s

Measured outcomes and boundaries

Required result	Current evidence
Local read-only benchmark execution (15 combinations, 1,800 requests)	PASS: 15/15 combinations pass; 0 errors (0.00% error rate); max p95 153.23 ms; peak throughput 1,016.85 req/s
SRS NFR-01 compliance (< 500 ms p95 at 100 concurrent users)	PARTIALLY EVIDENCED: Satisfied for local read queries at concurrency 1-10; 100-user staging test not yet executed
Concurrent check-in/out write operations and Neon DB lock contention	NOT EXECUTED - write benchmarks omitted to prevent staging data pollution
Multi-service integrated staging load test (100 and 300 concurrent users)	NOT EXECUTED - pending deployed cross-subsystem staging infrastructure

5. Food Ordering and Service Requests (FOSS)

Repository: forever-hotel/food-ordering | Branch: develop | Tested revision: c786e42 (merged PR #120 under Refs #119 / DDP-274; QA snapshot: 9ac1e6f3df9f93ee76be6db479dc8c4bf6529328) | CI date: 10 November 2026

Evidence: https://github.com/forever-hotel/food-ordering/pull/120, https://github.com/forever-hotel/food-ordering/actions/runs/42644489101, backend/test/perf/benchmark.spec.ts, backend/test/perf/benchmark-results.json

The committed FOSS QA Performance Benchmark suite (backend/test/perf/benchmark.spec.ts, PR #120 under Refs #119 / DDP-274) executes 8 automated concurrency scenarios against the 100% untouched production codebase in a strictly isolated local container and mock environment (127.0.0.1, zero remote Neon DB access). A total of 1,490 HTTP requests were executed with 0 errors (0.00% error rate), all achieving p95 < 500 ms and satisfying SRS NFR-01 constraints under concurrency levels between 25 and 100 virtual users. SC-PERF-02 incorporates 15 ms virtual-user pacing and SC-PERF-08 incorporates 10 ms cache-read pacing. Aggregate throughput across all 8 scenarios is 263.46 req/s (1,490 requests / 5.6555 s cumulative duration). Integrated multi-service staging load testing remains unverified.

Executed performance scenarios (FOSS automated benchmark suite)

Scenario	Operation / target endpoint	VUs / Reqs	p50 / p95 (ms)	Throughput & Gate	Outcome
SC-PERF-01	POST /guest-sessions/qr (QR Activation)	25 / 200	48.9 / 70.5	504.5 req/s (gate <300ms)	PASS
SC-PERF-02	GET /menu, /menu/windows (Menu Browse)	50 / 200	302.5 / 484.3	102.0 req/s (gate <500ms)	PASS
SC-PERF-03	POST /food-orders (Order Placement)	25 / 150	78.1 / 210.9	254.0 req/s (gate <500ms)	PASS
SC-PERF-04	GET /food-orders (Order Status & History)	80 / 240	116.5 / 155.6	697.0 req/s (gate <500ms)	PASS
SC-PERF-05	POST /service-requests (Service Request)	25 / 150	55.6 / 86.7	422.7 req/s (gate <500ms)	PASS
SC-PERF-06	POST /complaints (Complaint Submission)	25 / 150	73.6 / 93.3	355.7 req/s (gate <500ms)	PASS
SC-PERF-07	GET /realtime/events/poll (Realtime SSE)	100 / 300	93.3 / 140.6	916.8 req/s (gate <500ms)	PASS
SC-PERF-08	GET /menu (In-Memory Cache Reads)	25 / 100	216.6 / 473.4	79.4 req/s (gate <500ms)	PASS
TOTAL	Consolidated 8-Scenario Execution	100 max / 1,490	0.00% errors	263.46 req/s aggregate	PASS

Measured outcomes and boundaries

Required result	Current evidence
Automated benchmark suite execution (c786e42 / PR #120)	PASS: 1,490 requests across 8 automated scenarios executed in isolated local container; 100% 2xx status codes; 0 errors (0.00% error rate)
Throughput and latency distribution (empirical)	p50 = 48.9-302.5 ms, p95 = 70.5-484.3 ms (max p95 484.3 ms < 500 ms NFR-01 gate); aggregate throughput 263.46 req/s (1,490 reqs / 5.66 s)
SRS NFR-01 / NFR-03 / NFR-14 pass/fail determination	LOCAL BENCHMARK PASSED (all p95 < 500 ms at up to 100 VUs); multi-service staging cluster load testing remains pending

6. Kitchen Management System (KMS)

Repository: forever-hotel/kitchen-management | Branch: develop | Tested revision: 6d00047 (merged PR #108 image publication, PR #106 backend coverage >=85%, PR #105, PR #103) | CI date: 10 November 2026

Evidence: https://github.com/forever-hotel/kitchen-management/actions/runs/42645080201

KMS functional CI coverage expanded to 837 passing tests across 144 passing suites (70 backend unit suites [393 passed], 10 backend E2E suites [58 passed, 6 skipped], and 64 frontend Jest suites [386 passed]; CI Run 42645080201) with backend coverage reaching 91.89% stmts / 83.81% branches / 85.53% funcs / 94.00% lines (PR #106) and reviewed image publication checks passing (PR #108). However, no load, stress, KOT queue throughput, or cross-service FOSS-KMS-WKMS latency benchmark has been executed.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
KMS-P01	Menu read & category queries	Category/menu list p50/p95/p99, error rates, and DB query time	Not tested
KMS-P02	Incoming food-order burst	Order creation throughput, duplicate rejection, and DB lock performance	Not tested
KMS-P03	Concurrent kitchen status transitions	Concurrent chef status updates, atomicity, and outbound event emission	Not tested
KMS-P04	KOT print queue throughput	Print queue latency, retry handling, and duplicate prevention	Not tested
KMS-P05..06	FOSS-KMS-WKMS propagation & outage recovery	End-to-end timestamped event propagation and DB/broker recovery time	Not tested

Measured outcomes

Required result	Current evidence
Test environment, concurrency, and latency percentiles	Not measured (functional CI only: 837 passed, 6 skipped in Run 42645080201)
SRS pass/fail determination	Not evaluated

7. Worker Management System (WKMS)

Repository: forever-hotel/worker-management | Branch: develop | Tested revision: 1a33059c0005752ca709ca91ab11067dc6cff55a | CI date: 10 November 2026

Evidence: https://github.com/forever-hotel/worker-management/actions/runs/42504698971, https://github.com/forever-hotel/worker-management/actions/runs/42504970394

WKMS passes 649 automated unit/component tests (400 backend, 249 frontend) with >80% whole-application branch and line coverage on develop (1a33059c00), and local cross-service boundary checks with FDS (Phase B checkout consumer/escalation outbox and W04-B manual assignment) were demonstrated in bounded functional runs. Sustained load, concurrent task-claim race latency under 100-300 users, and Socket.IO fan-out benchmarks remain unexecuted.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
WKMS-P01	Worker task queue reads (GET /wkms/tasks/queue)	p50/p95/p99 latency and error rate under concurrent shift load	Not tested
WKMS-P02	Task claim concurrency (POST /wkms/tasks/:id/claim)	Max-three-tasks rule and double-claim lock contention under load	Not tested
WKMS-P03	Worker start / complete transitions	Transactional throughput, DB lock waits, and error rate	Not tested
WKMS-P04..05	RabbitMQ checkout intake & Socket.IO broadcast	FDS-to-WKMS event ingestion and task.created/claimed/escalated fan-out latency	Not tested
WKMS-P06	Shift summary dashboard (GET /wkms/shift/summary)	Response-time distribution under concurrent supervisor queries	Not tested

Measured outcomes

Required result	Current evidence
Test environment, concurrency, and latency percentiles	Not measured (functional CI only: 649 passed, 43 skipped in Run 42504698971)
SRS pass/fail determination	Not evaluated

8. Integrated performance test protocol

1. Deploy a version-locked staging release across all six subsystem backends/frontends, forever-hotel/api-gateway, forever-hotel/websocket-service, disposable PostgreSQL 16, and RabbitMQ using forever-hotel/infra.

2. Provision synthetic guest, receptionist, chef, worker, and manager accounts, dedicated room inventory (including an isolated single-room stay for HW last-room contention), and sandboxed payment/email/print stubs.

3. Record exact commit SHAs, container digests, CPU/RAM limits, PostgreSQL pool sizes, and dataset cardinalities prior to execution.

4. Execute the checked-in harnesses (HW performance/run.mjs, MAD backend/test/performance/run.ts, API Gateway npm run test:performance, FDS run-fds-performance.cjs extended with write flows, and FOSS benchmark.spec.ts / k6 scenarios SC-PERF-01..08) at graduated concurrency (1, 10, 50, 100, and 300 virtual users).

5. Archive raw JSON/CSV latency distributions (p50, p95, p99), throughput (RPS), HTTP/socket error rates, RabbitMQ consumer lag, and synchronized end-to-end event propagation traces before claiming NFR-01, NFR-03, or NFR-14 compliance.